EXP NO: 6
17/09/26

Hamming Code

Aim: write Program to implement error detection and correction using hamming code concept. Make test to run input data stream & verify Correction feature

Code:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if(2**r) >= (m+r+1):
            return r

def main():
    data = input("Enter data bits: ").strip()
    m = len(data)
    r = calc_redundant_bits(m)
    n = m+r
    Printf(f"Number of r needed : {r}")
    Printf(f"Total frame length : {n}")
    Code = ['0'] * (n)
    j = 0
    for i in range(1, n+1):
        if(i & (i-1)) != 0:
            Code[i] = data[-(j+1)]
            j += 1
```

```
for i in range(r):
    parity_pos = 2**i
    val = 0
    for pos in range(1, n+1):
        if (pos & parity_pos) != 0
            and pos != parity_pos:
            if code[pos] == '1':
                val ^= 1
    code[parity_pos] = str(val)
transmitted_str = "".join(code[1:][::-1])
Printf(f"Data to be transmitted: {transmitted_str}")
change = input("\n transmitted bit change").strip().upper()
if change == "yes":
    received_str = input(f"Enter corrupted data").strip()
    if (len(received_str) != n:
        Print(f'Error: Received data should be exactly {n} bits
            avg!")
        return
    received_code = ['0'] + list(received_str[::-1])
else:
    received_code = list(code)
```

```
error position = 0
for i in range (r):
    Parity_Pos = 2**i;
    Val = 0
    for Pos in range (1, n+1):
        if (Pos & Parity_Pos) != 0:
        if (received_Code [Pos] == '1'):
                Val ^= 1
    if val != 0:
        error_Position += Parity_Pos
if error_position == 0:
    Print ("\n Verification Result : No error detected")
else:
    Print ("\n Verification Result : Error detected!")
    Print (f "wrong bit at position : {error_Position}")
    received_Code {error_Position] = '0'
    if received_Code [error_Position] == '1' else '1'
Corrected_str = "".join (received_Code [1:] [::-1])
Print (f "Corrected data stream : {Corrected_str}")
if __name__ == "__main__":
        main ()
```

output:

① Enter data bits : 1011 00

Number of redundant bits (r) needed : 4

total frame length (m+r) : 10

Data to be transmitted : 1011 1000 11

Transmitted bit change ? (Yes/No) : yes

Enter the corrupted Data stream : 1011 1 0 1 011

Verification result : Error detected !

Wrong bit is at position : 4

Corrupted data streams : 1011 1 00011

② Enter data bits : 1011

Number of redundant bits (r) needed : 3

total data frame length (m+r) = 7

Data to be transmitted : 1 0 1 0 1 0 1

Transmitted bit change : (Yes/No) : No

Verification Result : No Error detection in transmission

Result : SW

Thus the Hamming Code was implemented Successfully for error detection and Correction.